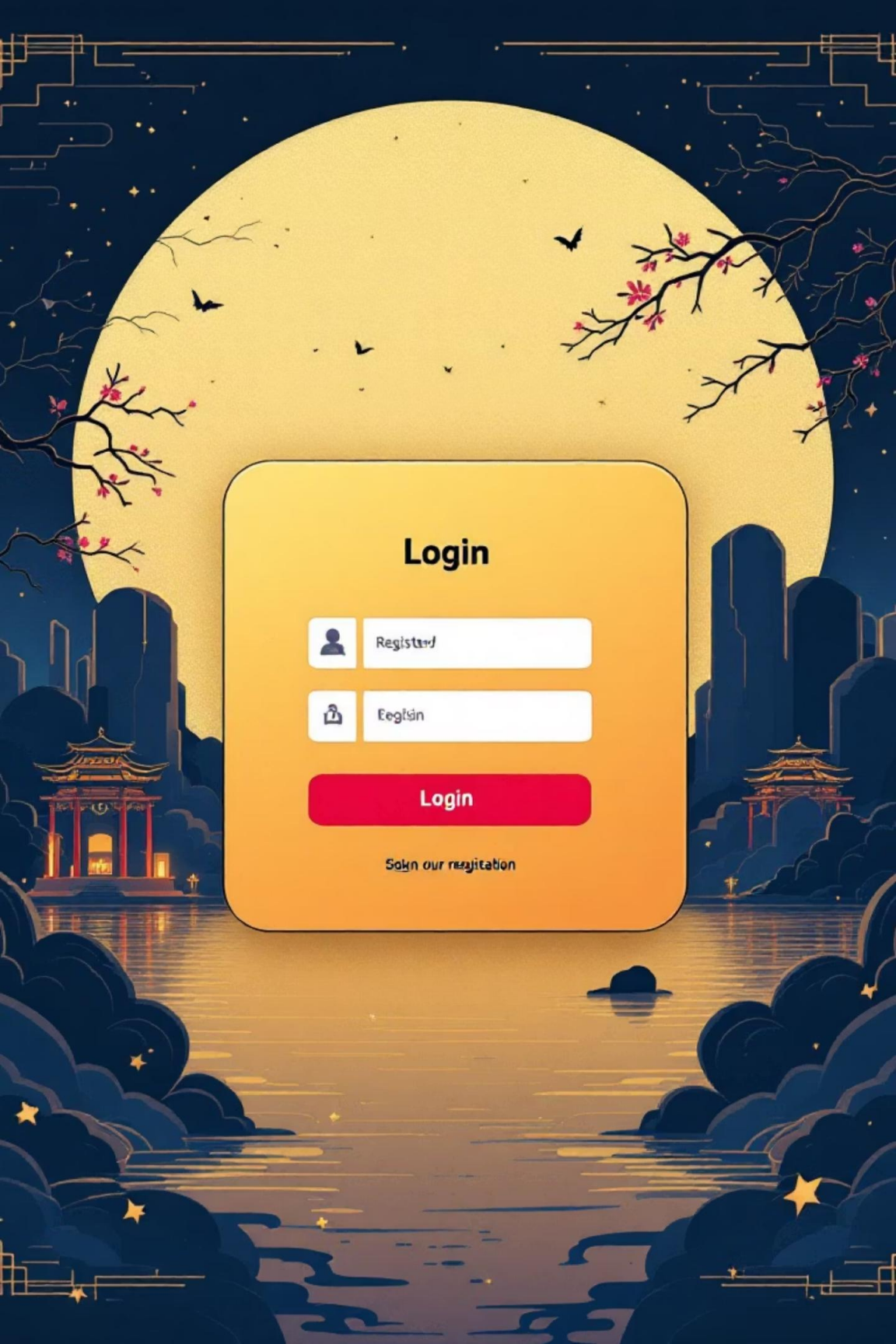


Complaint Management System

A comprehensive Java mini project demonstrating Swing GUI development, file handling capabilities, and object-oriented programming principles

Created by: Anirudh STP & K Hemanth





Login





Login

[Forgot our registration](#)

Project Overview



User Registration & Login

Secure authentication system allowing users to create accounts and access the platform with credential validation



Complaint Submission

Intuitive interface for users to submit detailed complaints with automatic tracking ID generation



Admin Review Panel

Dedicated dashboard for administrators to review, process, and update complaint statuses efficiently



File-Based Storage

Persistent data storage using text files with robust read/write operations for reliability

Learning Objectives



01

Master OOP Principles

Apply encapsulation, inheritance, and polymorphism in a real-world application context

02

Implement File Operations

Learn persistent storage techniques using Java's file handling capabilities

03

Handle Exceptions

Create and manage custom exceptions for robust error handling and validation

04

Explore Multithreading

Implement background operations to maintain responsive user interfaces

05

Build GUI Applications

Design intuitive Swing-based interfaces with professional layouts and components

Technologies Used



Java OOP

Core object-oriented programming using classes, objects, and interfaces to build modular, maintainable code



Abstraction

Abstract classes and interfaces defining contracts for file management and service operations



Text File Storage

Persistent data management using `BufferedReader` and `FileWriter` for reliable file operations



Inheritance & Polymorphism

Leveraging class hierarchies and method overriding to create flexible, extensible system components



Swing GUI Framework

Building interactive desktop interfaces with panels, buttons, text fields, and dialog boxes

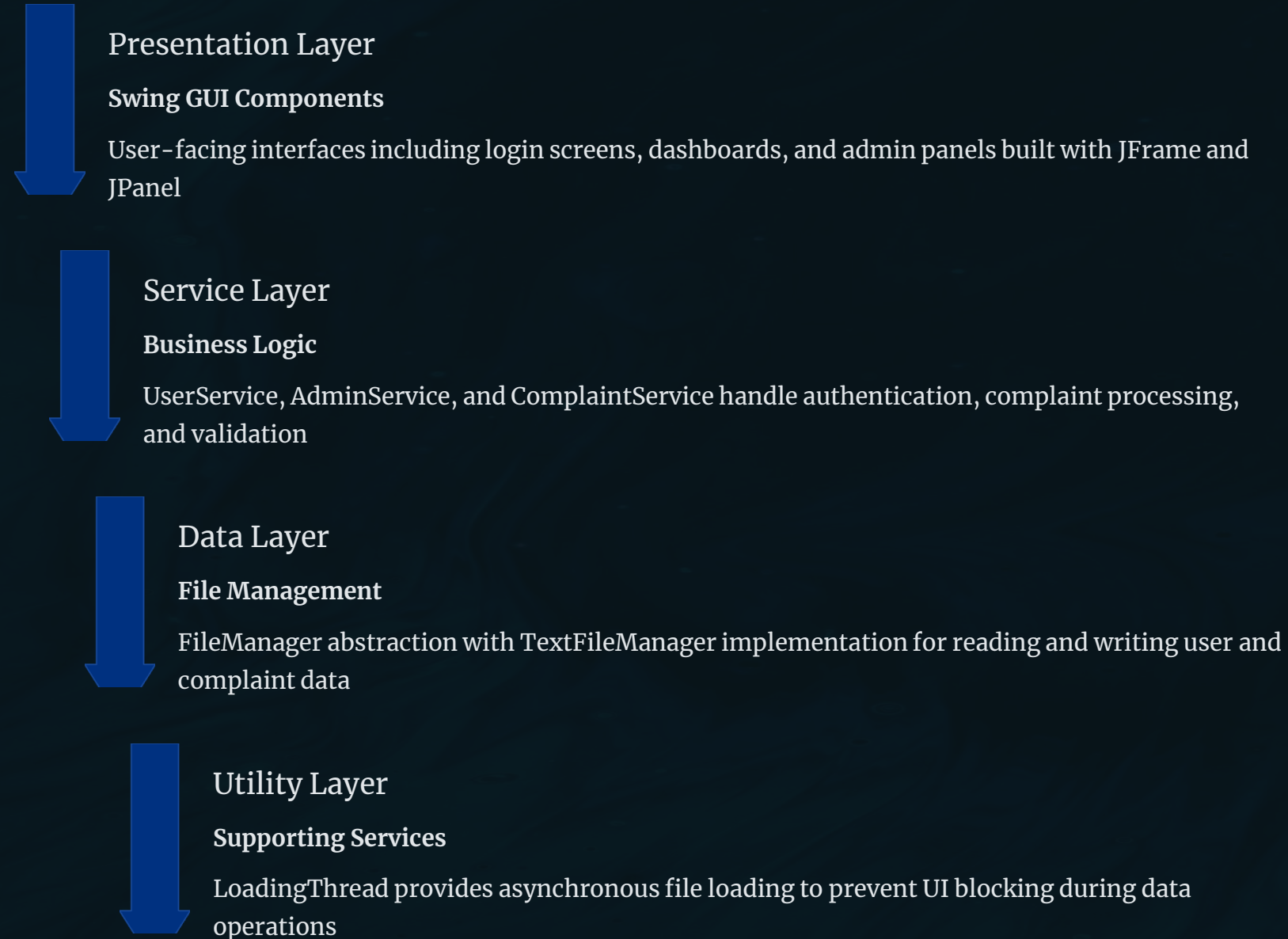


Multithreading

Concurrent execution for loading operations to ensure smooth, responsive user experience

System Architecture

The application follows a layered architecture pattern, separating concerns for maintainability and scalability. Each layer has distinct responsibilities and communicates through well-defined interfaces.



Core Classes



User

Represents system users with authentication credentials, storing username and encrypted password information



Complaint

Encapsulates complaint details including ID, description, submission date, status, and associated user information



UserService

Manages user registration, login validation, and credential verification with exception handling



AdminService

Provides administrative functions for viewing all complaints and updating complaint statuses



ComplaintService

Handles complaint submission, retrieval, and tracking with unique ID generation



FileManager

Abstract base class defining the contract for file operations with read and write methods



TextFileManager

Concrete implementation of FileManager handling text file I/O operations for users and complaints



LoadingThread

Background thread that loads data files asynchronously to maintain responsive GUI during startup

Class Hierarchy FlowChart



File Handling Strategy

Data Persistence

The system uses simple text files for storing user credentials and complaint records, providing a lightweight solution without database dependencies. This approach is ideal for learning file I/O fundamentals.

- **users.txt** – Stores username and password pairs with delimiter separation
- **complaints.txt** – Maintains complaint records with ID, description, status, and timestamps

File operations utilize **BufferedReader** for efficient reading and **FileWriter** for appending new records.



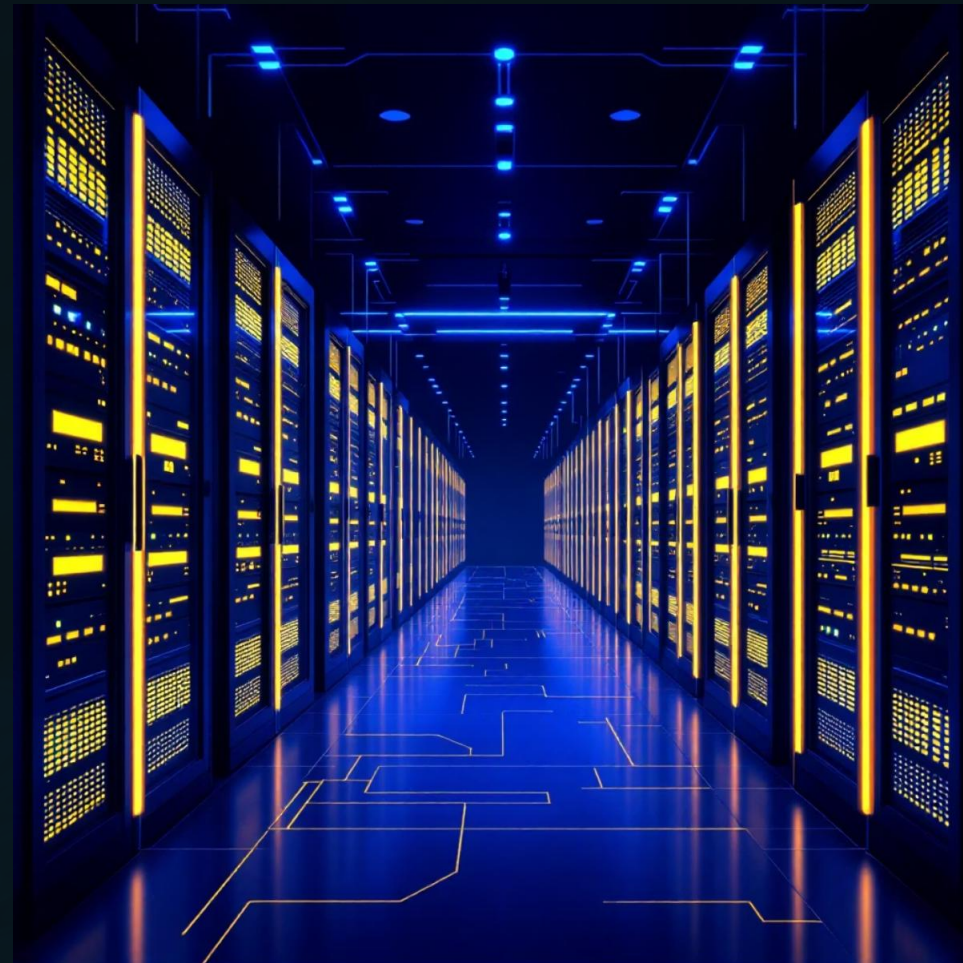
Abstract Layer

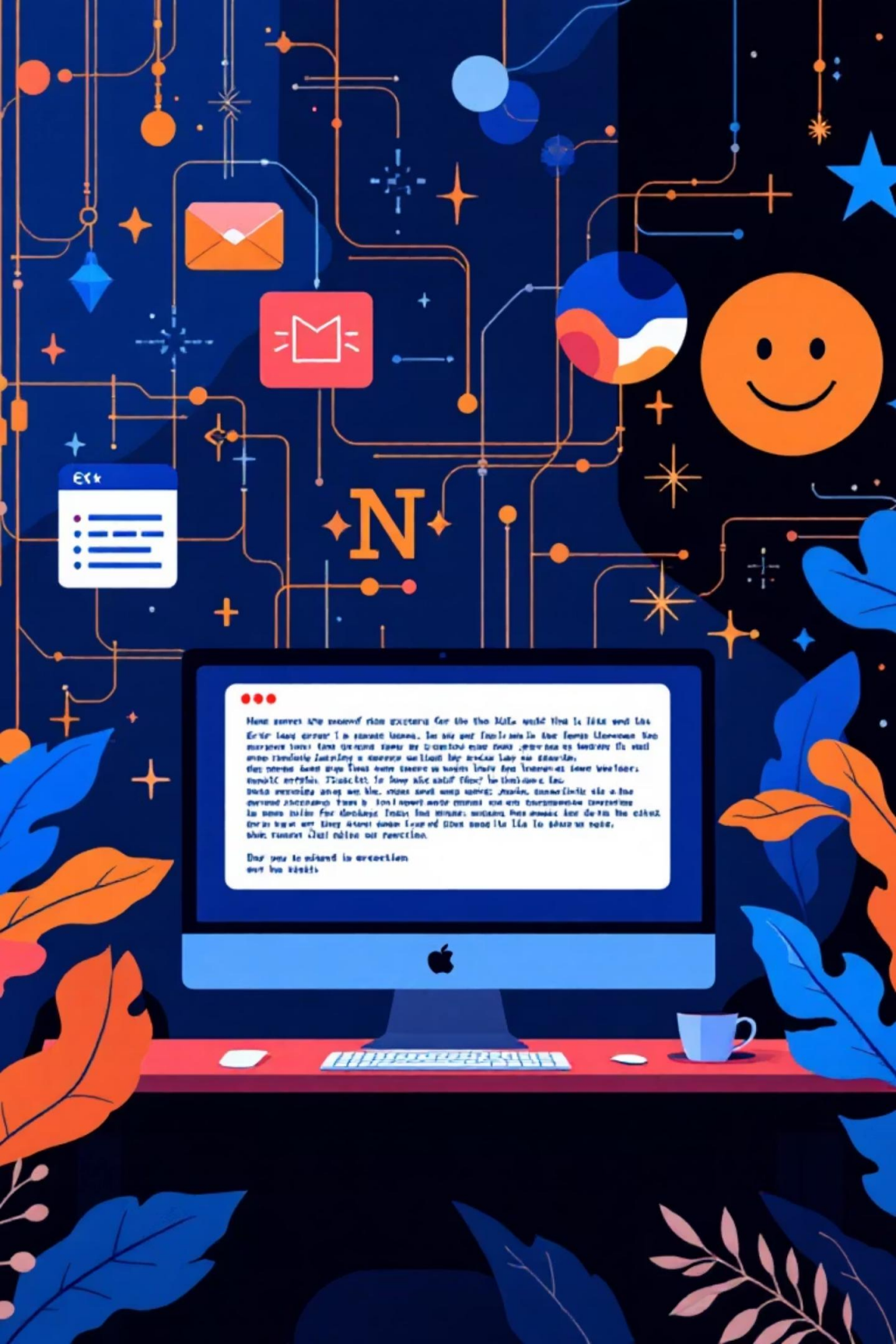
FileManager defines the interface for all file operations, promoting loose coupling



Concrete Implementation

TextFileManager implements specific logic for reading and writing text files





Custom Exception Handling

The system implements specialized exceptions to handle specific error scenarios gracefully, providing clear feedback and preventing application crashes. Each exception extends Java's Exception class.

DuplicateUserException

Thrown when attempting to register a username that already exists in the system

UserNotFoundException

Raised when login credentials reference a non-existent user account

IncorrectPasswordException

Triggered when the provided password doesn't match stored credentials

FileAccessException

Handles I/O errors during file read/write operations, ensuring robust data management

Multithreading Implementation

Background Loading

The **LoadingThread** class extends Thread to perform asynchronous file loading operations when the application starts. This prevents the GUI from freezing during data initialization.

Non-Blocking Operations

Files are loaded in the background while the main GUI thread remains responsive to user interactions

Silent Execution

The loading process runs without console output or progress indicators, providing a clean user experience

Thread Safety

Proper synchronization ensures data integrity when multiple threads access shared resources



Swing GUI Components

The interface follows a single-window paradigm where only one frame is visible at a time, creating a clean and focused user experience. Navigation between screens is handled through button actions and window disposal.

Login Screen

Entry point with username/password fields and authentication buttons

Registration Form

New user signup with credential validation and duplicate checking

User Dashboard

Main interface for submitting and viewing personal complaint history

Admin Panel

Privileged view for reviewing all complaints and updating statuses

📌 **Key Components:** JFrame, JPanel, JButton, JTextField, JTextArea, JOptionPane, JLabel, and layout managers for responsive design

Thank You

